

The Inference Economics Stack

Day 83 was the **demand** side of Agent FinOps — your routing, caching and budget levers. Day 84 was the **people** who run the fleet. Today is the **supply** side: the silicon and serving layer that actually sets the cost-per-token your FinOps levers push against. As frontier models compress to a near-tie (within ~3%), you can't out-model a rival — so the margin war moves down the stack to **who can serve a token cheapest**. And the numbers moved hard this month: AWS shipped the 192-core **Graviton5** on June 15, Amazon's custom-silicon run-rate crossed **\$20B**, and SpaceX's S-1 revealed Anthropic is paying xAI **\$1.25B a month** to rent 300MW of compute. This issue walks the three layers that set cost-per-token — silicon, serving engine, optimisation — and why the GPT-4-class token just fell ~**1,000x** in three years.

~\$0.40 / M cost of a GPT-4-class output token today — a ~1,000x collapse in 3 yrs (from ~\$30/M in 2023)	>\$20B Amazon custom-silicon run-rate (Graviton+Trainium+Nitro); ~\$50B if sold externally	55–80% of enterprise AI GPU spend goes to INFERENCE, not training — the bill that never stops	\$1.25B / mo Anthropic pays xAI to rent 300MW / 220K GPUs through May 2029 (SpaceX S-1)
---	---	---	---

1 · Why the supply side is suddenly the battleground

Day 83 gave you the demand-side levers — model routing, caching, budget caps, tagging. Every one of those pushes against a floor that something else sets: the **cost-per-token** of actually running the model. That floor is the supply side, built from three stacked layers — the **silicon** the model runs on, the **serving engine** that schedules the work, and the **optimisation techniques** (quantisation, speculative decoding) that squeeze more tokens out of each chip-second. Your FinOps savings are *bounded* by how cheap this stack can make a token; the cheaper the floor, the more headroom your routing and caching buy you.

Two things make this the battleground of mid-2026. First, model performance has compressed to a near-tie — the top models sit within ~3% — so you can't win on capability; you win on **unit economics**. Second, the spend has shifted decisively: analysts now put **55–80% of enterprise AI GPU budget on inference**, not training. Training is a one-time capital event; serving is a bill that accrues every hour, for the life of the product. Whoever owns the cheapest path from prompt to token owns the margin — which is why Amazon, Google, NVIDIA, AMD and a wave of start-ups are pouring tens of billions into custom silicon aimed squarely at the decode phase.

***The headline proof this is now a balance-sheet fight:** SpaceX's S-1 disclosed that Anthropic is paying xAI **\$1.25B a month** to rent 300MW of compute (220,000 NVIDIA GPUs at Colossus 1 near Memphis) through May 2029 — over \$40B from one frontier lab to a direct rival, capped at 11% of the park, terminable on 90 days' notice. When labs rent compute from competitors at that scale, the binding constraint on agentic AI is no longer the model — it's the inference substrate under it.*

2 · The silicon layer — custom chips break the GPU monopoly

For three years inference meant NVIDIA GPUs. In 2026 the hyperscalers' custom silicon turned that into a real contest — because designing a chip for the **decode phase** (memory-bound, latency-sensitive) rather than for training (compute-bound) changes the price-performance maths. AWS shipped the 192-core **Graviton5** on June 15 (5x larger cache, ~35% faster ML inference, M9g up to 25% faster), but **Trainium** is the bigger story: Trainium2 runs ~30% better price-performance than comparable GPUs and is largely sold out; Trainium3 is another 30–40% better and nearly fully subscribed. Amazon's custom-silicon run-rate has crossed **\$20B**, and Anthropic has committed >\$100B to AWS over ten years and up to 5GW of Trainium capacity.

The chip	What it changes for inference	Status
AWS Graviton5	192-core CPU, 5x cache, ~35% faster ML inference; M9g +25% vs prior gen; pitched for the agentic era	GA June 15 2026
AWS Trainium2 / 3	~30% then +30–40% better price-perf vs GPUs; >\$225B revenue commitments; Anthropic up to 5GW	Trn2 sold out; Trn3 ~full
Google Ironwood → 8t/8i	“First TPU for the age of inference”; 8i is a dedicated inference chip, +80% price-perf, TSMC 2nm	Ironwood GA; 8t/8i previewed
NVIDIA Rubin	Next-gen memory + networking gains that shift TCO; the incumbent's answer	Ramp ~Q3 2026 (slipped a Q)
AMD MI400	432GB HBM4, 19.6 TB/s bandwidth; aimed at the largest serving clusters	Hyperscalers first; broad 2027
Groq 3 LPU	~35x more inference throughput <i>per megawatt</i> than HBM GPUs; attacks the decode phase directly	Announced GTC 2026

Google reframed the category: Ironwood went GA as the TPU “for the age of inference,” and Cloud Next 2026 previewed an eighth generation *split* into a training chip (8t) and a dedicated inference chip (8i). The bifurcation is the tell — general-purpose silicon is giving way to chips purpose-built for serving. For an agent fleet the practical point is that “**which chip**” is now a **cost lever you can pull**: the same model on Trainium3, a TPU 8i or a Groq LPU can carry a materially different cost-per-token than the default NVIDIA path.

3 · The serving engine — software that 5–8x's your throughput

Silicon sets the ceiling; the **serving engine** decides how much of it you actually use. The three that matter in 2026 serve the same purpose but make different bets — and the rule of thumb is simple.

Engine	Pick it for	Signature primitive
vLLM	Fastest path to production; broad model + hardware support	PagedAttention (KV-cache paging) + continuous batching
TensorRT-LLM	Peak throughput on NVIDIA hardware	Kernel-fused, in-flight batching tuned for CUDA
SGLang	Heavy multi-turn / prefix-reuse agent workloads	RadixAttention prefix caching + disaggregated prefill

Three primitives separate a cheap deployment from an expensive one. Continuous batching admits a new request the instant a slot frees (the biggest GPU-utilisation win of the last two years). Prefix caching reuses the KV cache for shared prompt prefixes — decisive for agents, whose long system prompts and tool schemas repeat on every call (the serving-side mirror of the Day 83 caching lever). Disaggregated prefill splits the compute-bound prefill and memory-bound decode onto separate devices so they stop interfering — now standard in vLLM and SGLang. Turning these on is often a bigger win than swapping the model.

4 · The optimisation layer — where the \$0.40 token comes from

The last layer is the math trick that collapsed the price: serve the model in **lower precision** and predict **more tokens per pass**. Quantisation is the workhorse — moving FP16→FP8 roughly halves the cost of an 8x-H100 node at negligible quality loss; 4-bit (AWQ) cuts VRAM ~75%, letting the same model fit on a cheaper GPU class entirely. Speculative decoding adds throughput from the other side: a small draft model proposes tokens and the large model verifies them in parallel.

Technique	What it does	The win
FP8 quantisation	Half-precision serving, negligible quality loss for most production traffic	~\$1.90 → ~\$0.95–1.10 / M
4-bit (AWQ)	Cuts VRAM ~75% — same model fits a cheaper GPU class	Drops a hardware tier
Speculative decoding	Draft model proposes, target verifies in parallel; output-heavy work	2–4x faster, no quality cost
Stacked (FP8+FA3+batch+spec)	All four together on an H100 vs naive FP16 + static batching	5–8x better cost-efficiency

*The discipline that ties it back to Day 83: cheaper tokens aren't the goal — cheaper successful outputs are. The FinOps Foundation warns that instrumenting cost-per-token without a quality gate is a trap: a 4-bit model that fails 5% more tasks can cost MORE per completed job than the FP8 version it replaced. The supply-side stack sets your floor, but you still measure the win in **cost-per-successful-task** on the same OTEL→WORM stream you built for the regulator (Day 81), the buyer (Day 82) and your FinOps dashboard (Day 83).*

5 · Viral app spotlight — llama.cpp, the engine that drives cost-per-token to ~\$0

On a day about who serves a token cheapest, the trending project is the one that drives the price to the floor. **llama.cpp** (Georgi Gerganov / ggml-org) crossed **100,000 GitHub stars in March 2026** — the fastest open-source AI project ever to reach the mark, quicker than PyTorch (~7 years) or TensorFlow (~8 years). It's the C/C++ inference engine that quietly powers most of the local-AI world (Ollama, LM Studio and dozens of apps sit on top of it), and its momentum is staggering: 700+ contributors merged **3,800+ pull requests in 2025**, roughly 3x the PR throughput of NVIDIA's fully-funded TensorRT-LLM. One GGUF model runs across Apple Metal, NVIDIA CUDA, AMD ROCm, Intel SYCL, Vulkan and ARM NEON — from a Raspberry Pi 5 to an 8x-GPU server — and now ships WebGPU inference in the browser. Why it matters here: local inference on llama.cpp costs roughly **\$0.002 per million tokens in electricity**, versus \$2.50–15.00/M through cloud APIs. That's the cost-per-token war reaching the desktop — the cleanest proof that the supply side, not the model, is where the economics are now won. (OpenClaw still tops the raw OSS star charts at 210K+ as the local-first foil — viral, but a personal agent, not the serving engine underneath it.)

100K stars fastest OSS AI project ever to 100K (Mar 2026) — beat PyTorch & TensorFlow	~\$0.002 / M local inference in electricity vs \$2.50–15.00/M cloud API — the floor	RPi 5 → 8x GPU one GGUF model across Metal / CUDA / ROCm / SYCL / Vulkan / ARM + WebGPU
---	---	---

Practical takeaways

1. Treat the serving stack as a product decision, not an afterthought.

Pick the engine for your workload — vLLM for fastest-to-prod, TensorRT-LLM for peak NVIDIA throughput, SGLang for prefix-heavy agents — then turn on continuous batching, prefix caching and disaggregated prefill. For repetitive agent prompts that is often a bigger cost win than swapping the model, and it's entirely under your control.

2. Quantise to FP8 by default — and measure cost-per-successful-task.

FP8 roughly halves cost-per-token at negligible quality loss; 4-bit can drop you to a cheaper GPU class; speculative decoding adds 2–4x on output-heavy work. But validate on your golden set — a lower-precision model that fails more tasks can cost MORE per completed job. The metric is cost-per-successful-output, not cost-per-token (Day 83).

3. Make “which silicon” a lever you actually pull.

The same model on Trainium3, a TPU 8i, a Groq LPU or llama.cpp on your own hardware can carry a very different cost-per-token than the default NVIDIA path. Benchmark at least one non-NVIDIA option per workload — and for low-volume or privacy-bound tasks, remember local inference runs at ~\$0.002/M in electricity versus \$2.50–15/M in the cloud.

***Tomorrow (Day 86):** Agent Interop & the Protocol Stack 2.0 — one year on from MCP. How the connective tissue settled: MCP at ~97M monthly SDK downloads, A2A v1.x under the Linux Foundation with 150+ production orgs, and AG-UI converging on a shared event model — the standards layer that lets today's cheap-token fleet actually talk to each other.*